

Smart Resume Crafter with Real-Time AI Suggestions & Live Preview

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

BY

Tanuj Deshmukh (SC23CS302036)

Rabjyot Singh Rajpal (SC23CS302031)

Under the Guidance of

Dr. Rohit Gupta

Dr. Shrikant Telang



Department of Computer Science

Faculty of Science

MEDICAPS UNIVERSITY, INDORE- 453331

MAY-2026

Report Approval

The project work “**Smart Resume Crafter with Real-Time AI Suggestions & Live Preview**” is hereby approved as a creditable study of a science subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

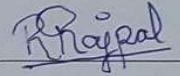
We hereby declare that the project entitled "Smart Resume Crafter with Real-Time AI Suggestions & Live Preview" submitted in partial fulfillment for the award of the degree of Bachelor of Science in 'Computer Science' completed under the supervision of **Dr. Rohit Gupta and Dr. Shrikant Telang**, Department of Computer Science & Engineering, Faculty of Engineering, Mediacaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature: 

Name: Tanuj Deshmukh (SC23CS302036)

Date: April 2026

Signature: 

Name: Rabjyot Singh Rajpal (SC23CS302031)

Date: April 2026

Certificate

We, **Dr. Rohit Gupta & Dr. Shrikant Telang** certify that the project entitled “**Smart Resume Crafter with Real-Time AI Suggestions & Live Preview**” submitted in partial fulfillment for the award of the degree of Bachelor of Science by **Rabjyot Singh Rajpal & Tanuj Deshmukh** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Rohit Gupta

Dr. Shrikant Telang

Department of Computer Science & Engineering

Medicaps University, Indore

Prof. (Dr.) Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm have always boosted up my morale. Furthermore, we thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean of Engineering, Mediacaps University, for giving me a chance to work on this project. We would also like to thank our Head of the Department **Prof. (Dr.) Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We sincerely thank our project guides, **Dr. Rohit Gupta**, **Dr. Shrikant Telang**, for the expert guidance, valuable suggestions, and continued motivation throughout the development of this project. The timely feedback and technical advice provided helped me navigate through challenges and deliver a quality outcome.

We would also like to express our gratitude to all the faculty members of the Department of Computer Science & Engineering, Mediacaps University, for their direct or indirect support and academic inspiration. Special thanks to our fellow students and classmates for their cooperation, ideas, and moral support throughout this project.

Finally, we thank our family for their unconditional support and encouragement throughout my academic journey. This project would not have been possible without their love and patience.

Name: Tanuj Deshmukh (SC23CS302036)

Name: Rabjyot Singh Rajpal (SC23CS302031)

B.Sc. III Year (A)

Department of Computer Science

Faculty of Science

Abstract

The Luminary AI – Smart Resume Builder with Real-Time AI Suggestions & Live Preview is a full-stack web application designed to revolutionize the resume creation process by leveraging artificial intelligence and modern web technologies. In today's competitive job market, a well-crafted resume is often the first point of contact between a candidate and a potential employer. However, many job seekers — particularly students and early-career professionals — struggle with effectively articulating their skills, experiences, and achievements in a professional format. This project addresses that challenge by providing an intelligent, automated, and user-friendly platform for building high-quality resumes.

The system follows a multi-step guided form approach, allowing users to input structured information across comprehensive sections including Personal Details, Professional Summary, Work Experience, Education, Skills, Projects, Certifications, and Research Papers. A real-time live preview panel renders the resume side-by-side as the user fills in data, eliminating the traditional edit-save-review cycle and providing instant visual feedback.

At its core, the application integrates with the OpenRouter API powered by Google's Gemini 3.1 Pro large language model to deliver AI-generated content suggestions. Users can generate professionally phrased summaries and work experience descriptions with a single click, choosing from multiple AI-suggested options tailored to their job title and professional context. This AI-driven approach significantly enhances the content quality of resumes while reducing the time and effort required.

The project is built on a full-stack MERN architecture — MongoDB for data persistence, Express.js for RESTful API services, React.js (Vite) for a dynamic and responsive single-page frontend, and Node.js as the server runtime. Tailwind CSS is used for rapid, utility-first UI styling, ensuring a visually polished and modern interface. User authentication is implemented using bcrypt for password hashing and JSON Web Tokens (JWT) for secure session management. The application also supports customizable theme colors and a PDF export feature, making the final output immediately usable for job applications.

The system prioritizes user-friendliness, automation, and customization, making professional resume creation accessible to users regardless of their design or writing expertise. By combining

the power of large language models with an intuitive editing experience, the Luminary AI Resume Builder demonstrates a practical and impactful application of AI in everyday career development tools.

Keywords:

Resume Builder, Artificial Intelligence, Large Language Models, React.js, MongoDB, OpenRouter API, Gemini AI, Live Preview, PDF Export, Full-Stack Web Application, MERN Stack

Table of Contents

Chapter No.	Topics	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi-vii
	Table of Contents	viii- x
	List of figures	xi
	List of tables	xii
	Abbreviations	xiii
	Notations & Symbols	xiv
Chapter 1	Introduction	1-4
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	3
	1.6 Source of Data	3
	1.7 Chapter Scheme	4
Chapter 2	REQUIREMENTS SPECIFICATION	5-10
	2.1 User Characteristics	5

	2.2 Functional Requirements	6
	2.3 Dependencies	7
	2.4 Performance Requirements	8
	2.5 Hardware Requirements	8
	2.6 Constraints & Assumptions	9
Chapter 3	Design	11-21
	3.1 Algorithm	11
	3.2 Function Oriented Design for procedural approach	11
	3.3 System Design	12
	3.3.1 Data Flow Diagrams	13
	3.3.2 Activity Diagram	15
	3.3.3 Flow Chart	16
	3.3.4 Class Diagram	17
	3.3.5 ER Diagram	18
	3.3.6 Sequence diagram	19
	3.4 Database Design	20
	3.4.1 Logical Database Design	20
	3.4.2 Physical Database Design	21
Chapter 4	Implementation, Testing, and Maintenance	22-25
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	22
	4.2 Testing Techniques and Test Plans	23
	4.3 Installation Instructions	24

	4.4 End User Instructions	25
Chapter 5	Results and Discussions	26-33
	5.1 User Interface Representation	26
	5.2 Brief Description of Various Modules of the system	26
	5.3 Snapshots of system with brief detail of each	27
	5.4 Back Ends Representation	30
	5.5 Snapshots of Database Tables with brief description	32
Chapter 6	Summary and Conclusions	34-35
Chapter 7	Future scope	36
	Appendix	37-39
	Bibliography	40-41

List of Figures

Figure No.	Figure Title	Page No.
Figure 3.3.1	Data Flow Diagram (Level 0)	13
Figure 3.3.2	Data Flow Diagram (Level 1)	14
Figure 3.3.3	Activity Diagram	15
Figure 3.3.4	Flow Chart	16
Figure 3.3.5	Class Diagram	17
Figure 3.3.6	ER Diagram	18
Figure 3.3.7	Sequence Diagram	19
Figure 5.1	The Landing Page	27
Figure 5.2	Login Page	28
Figure 5.3	Dashboard	28
Figure 5.4	The Split-Screen Editor	29
Figure 5.5	AI Summary Generation	29
Figure 5.6	Theme Customization	30
Figure 5.7	MongoDB Database Structure	31
Figure 5.8	MongoDB Resume Collection	33
Figure 5.9	MongoDB User Collection	33

List of Tables

Table No.	Table Title	Page No.
Table 2.1	User Characteristics	5
Table 2.2	Functional Requirements	7
Table 2.3	Hardware and Software Requirements	9
Table 3.1	Core Modules and Responsibilities	12
Table 4.1	Tools and Technologies Used	23
Table 4.2	Testing Summary	24
Table 5.1	User Interface Components	26
Table 5.2	Database and local storage Structure	32

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
UI	User Interface
UX	User Experience
PDF	Portable Document Format
LLM	Large Language Model
ODM	Object Data Modeling
JWT	JSON Web Token
CSS	Cascading Style Sheets
HTML	HyperText Markup Language

Notations & Symbols

Symbol	Meaning
_id	Unique identifier in MongoDB
→	Data flow direction
[]	Array/List structure
{}	Object (JSON structure)
%	Percentage
ms	Milliseconds
GB	Gigabytes

Chapter 1 – Introduction

1.1 Introduction

The process of crafting a professional resume is a critical step for students, recent graduates, and seasoned professionals alike as they navigate the highly competitive job market. A well-structured resume serves as the first point of contact between a candidate and a prospective employer, often determining whether the candidate will proceed to the interview stage. However, many individuals struggle with formatting, phrasing, and organizing their skills effectively, leading to subpar representations of their true capabilities.

To address this challenge, the "**Smart Resume Crafter with Real-Time AI Suggestions & Live Preview**" (also referred to as Luminary AI Resume Builder) has been developed. This web-based application leverages modern frontend technologies and Artificial Intelligence (AI) to provide a seamless, intuitive, and highly automated resume-building experience. By integrating the OpenRouter API (utilizing advanced language models like Gemini), the system can intelligently generate professional summaries, suggest impactful bullet points for work experience, and optimize the overall content of the resume. Furthermore, the application features a side-by-side live preview, allowing users to instantly visualize their document as they input their details. This eliminates the guesswork associated with traditional word processors and ensures a polished, industry-standard final product.

1.2 Literature Review

The evolution of resume building has transitioned from manual typesetting and basic word processors to sophisticated, template-driven online platforms.

- **Traditional Word Processors (e.g., Microsoft Word, Google Docs):** While these tools offer maximum flexibility, they require significant manual effort to maintain consistent formatting. Users often face issues with misaligned text, broken layouts, and time-consuming design adjustments.
- **Template-Based Builders (e.g., Canva, Zety):** These platforms introduced predefined templates, significantly improving the visual appeal of resumes. However, they largely rely on the user to generate the actual content, offering little to no guidance on professional phrasing or keyword optimization.

- **Early AI-Assisted Tools:** Recently, some platforms have integrated basic AI to check spelling and grammar (e.g., Grammarly). However, these tools do not natively generate context-aware professional content tailored to specific job roles.

The proposed system bridges the gap between aesthetic design and content generation. Unlike existing systems that only offer visual templates, the Smart Resume Crafter actively assists in content creation. By utilizing Large Language Models (LLMs), the system can generate customized summaries and experience descriptions, reducing the cognitive load on the user while significantly improving the quality of the output.

1.3 Objectives

- The primary objectives of this project are as follows:
- **Automated Content Generation:** To integrate AI capabilities that can generate professional resume summaries and job experience descriptions based on minimal user input (e.g., job title and key skills).
- **Real-Time Visualization:** To implement a live preview feature that instantly updates the visual representation of the resume as the user types, ensuring immediate feedback on layout and design.
- **User-Friendly Interface:** To design a highly intuitive, responsive, and modern user interface using React.js and Tailwind CSS, providing a seamless experience across both desktop and mobile devices.
- **Standardized Exporting:** To enable users to effortlessly export their completed resumes in a widely accepted format (PDF) without losing any styling or formatting.
- **Data Persistence:** To securely manage user data and resume states during the session, allowing users to edit and refine their resumes without data loss.

1.4 Significance

The significance of this project lies in its ability to democratize access to high-quality resume creation. Professional resume writing services are often expensive and inaccessible to students

or entry-level job seekers. By providing an intelligent, automated, and free-to-use platform, this project empowers individuals to create compelling resumes that pass Applicant Tracking Systems (ATS) and catch the eye of human recruiters. Furthermore, the integration of AI reduces the time required to draft a resume from hours to mere minutes, increasing overall productivity.

1.5 Research Design

The research and development of this project followed an iterative Software Development Life Cycle (SDLC) model, specifically the Agile methodology.

- **Requirement Analysis:** Gathering user pain points related to traditional resume building through surveys and analysing existing solutions.
- **System Design:** Designing the architecture of the web application, selecting the appropriate technology stack (Vite, React.js, Tailwind CSS), and planning the API integration (OpenRouter).
- **Development:** Implementing the frontend components, setting up state management for the live preview, and writing the logic for AI prompt generation.
- **Testing & Refinement:** Conducting usability testing to ensure the interface is intuitive and refining the AI prompts to yield consistently high-quality results.

1.6 Source of Data

- The data utilized in this project primarily consists of:
- **User Input:** Personal details, educational background, work experience, and skills provided by the user through dynamic forms.
- **AI Generated Data:** Textual content (summaries, bullet points) generated dynamically via the OpenRouter API based on the context provided by the user.
- **Design Assets:** Standardized layout metrics, font pairings, and color palettes defined within the system's CSS framework.

1.7 Chapter Scheme

The report is organized into the following chapters:

- **Chapter 1: Introduction** - Provides an overview of the project, its objectives, significance, and literature review.
- **Chapter 2: Requirements Specification** - Details the functional and non-functional requirements, hardware/software needs, and constraints.
- **Chapter 3: Design** - Covers the system architecture, flowcharts, class diagrams, and database design.
- **Chapter 4: Implementation, Testing, and Maintenance** - Discusses the technologies used, coding standards, and testing methodologies applied.
- **Chapter 5: Results and Discussions** - Showcases the final user interface, module descriptions, and system snapshots.
- **Chapter 6: Summary and Conclusions** - Summarizes the project outcomes and final thoughts.
- **Chapter 7: Future Scope** - Outlines potential future enhancements for the application.

Chapter 2 – Requirements Specification

2.1 User Characteristics

The target audience for the Smart Resume Crafter is broad, encompassing anyone seeking employment or professional advancement.

- **Students/Recent Graduates:** Users with limited professional experience who require assistance in framing their academic achievements and soft skills professionally. They need an intuitive interface that guides them step-by-step.
- **Mid-to-Senior Professionals:** Users looking to update their existing resumes quickly. They require advanced customization options and the ability to generate highly specific, industry-relevant summaries.
- **Technical Proficiency:** The system is designed to require zero technical expertise. Users only need basic web browsing skills to navigate the forms and download their final PDF.

Table 2.1: User characteristics

User Type	Characteristics & Needs
Students / Recent Graduates	Have limited professional experience. They require assistance in framing their academic achievements and soft skills professionally. They need an intuitive, step-by-step interface.
Mid-to-Senior Professionals	Looking to update existing resumes quickly. They require advanced customization options and the ability to generate highly specific, industry-relevant AI summaries.
General Users	Require zero technical expertise. Users only need basic web browsing skills to navigate the input forms and download their final PDF.

2.2 Functional Requirements

Functional requirements define the specific behaviors and capabilities the system must possess:

- **User Input Forms:** The system must provide structured, dynamic forms for users to input Personal Details, Summary, Professional Experience, Education, and Skills.
- **Live Preview Engine:** The system must render a real-time visual representation of the resume alongside the input forms. Any change in the form must instantly reflect in the preview.
- **AI Summary Generation:** The system must allow users to generate a professional summary by clicking an "Add from AI" button. The system should fetch data from the OpenRouter API and present three distinct summary options based on the user's job title.
- **Dynamic Sections:** Users must be able to add, edit, or delete multiple entries for Experience, Education, and Skills dynamically.
- **Theme Customization:** The system must allow users to change the primary theme colour of their resume, updating the visual preview instantly.
- **PDF Export:** The system must provide a mechanism to download the generated resume as a high-quality, perfectly formatted PDF document.

Table 2.2: Functional requirements

Req. ID	Feature	Description
FR-01	User Input Forms	System provides structured, dynamic forms for Personal Details, Summary, Professional Experience, Education, and Skills.
FR-02	Live Preview Engine	System renders a real-time visual representation of the resume. Changes in the form reflect instantly in the preview.
FR-03	AI Summary Generation	System integrates with the OpenRouter API to automatically generate three distinct professional summary options based on the user's job title.
FR-04	Dynamic Sections	Users can dynamically add, edit, or delete multiple entries for Experience, Education, and Skills.
FR-05	Theme Customization	Users can change the primary theme color of their resume, updating the visual aesthetic instantly.
FR-06	PDF Export	System provides a mechanism to download the generated resume as a perfectly formatted, high-quality PDF document.

2.3 Dependencies

- **External APIs:** The core AI functionality is entirely dependent on the availability and responsiveness of the OpenRouter API (or specific LLMs like Gemini).
- **Browser Compatibility:** The application relies on modern web browsers (Chrome, Firefox, Edge, Safari) that support ES6 JavaScript, CSS Grid/Flexbox, and the Web Print API for PDF generation.
- **Network Connection:** An active internet connection is required to utilize the AI generation features and to load external assets (like Google Fonts).

2.4 Performance Requirements

- **Response Time:** The live preview must update within 100 milliseconds of user input to ensure a seamless experience.
- **API Latency:** The AI content generation should ideally return results within 2 to 5 seconds to prevent user frustration. Loading states/spinners must be displayed during this wait time.
- **Scalability:** The frontend architecture should be lightweight, ensuring fast load times (under 3 seconds for the initial page load) even on slower network connections.

2.5 Hardware Requirements

Client-Side (User):

- **Device:** Any desktop, laptop, or modern tablet/smartphone.
- **Processor:** Intel Core i3 / AMD Ryzen 3 or equivalent (minimum).
- **RAM:** 4 GB RAM (minimum).
- **Storage:** Negligible (Web-based application).

Server-Side (Development/Hosting):

- **Processor:** Multi-core processor (e.g., Intel Core i5 or higher).
- **RAM:** 8 GB RAM (minimum for running Node.js development server).
- **Storage:** 256 GB SSD.

2.6 Constraints & Assumptions

Table 2.3: Hardware and software requirements

Component	Client-Side (User Requirement)	Server-Side (Development Requirement)
Processor	Intel Core i3 / AMD Ryzen 3 (minimum)	Multi-core processor (Intel Core i5 or higher)
RAM	4 GB RAM (minimum)	8 GB RAM (minimum for Node.js server)
Storage	Negligible (Web-based application)	256 GB SSD
Operating System	Any OS with a modern browser	Windows, macOS, or Linux
Web Browser	Chrome, Firefox, Edge, Safari	N/A
Network	Active connection (for AI features)	Active connection (for APIs and dependencies)

- **Assumptions:**

- It is assumed that the user will provide accurate and truthful information regarding their professional background.
- It is assumed that the user has a stable internet connection while utilizing the AI features.

- **Constraints:**

- **API Rate Limits:** The usage of the OpenRouter API may be subject to rate limiting or token costs, which constrains how frequently a single user can request AI generation in a short time frame.

- **PDF Formatting Limitations:** Rendering HTML to PDF via browser print functions can sometimes yield minor inconsistencies across different browsers (e.g., Firefox vs. Chrome), necessitating careful CSS optimization.
- **Screen Size:** While responsive, the side-by-side editing and live preview experience is highly constrained on small mobile screens, requiring a stacked layout approach for smaller viewports.

Chapter 3 – Design

3.1 Algorithm

The core intelligence of the Smart Resume Crafter lies in the algorithm used to fetch, process, and render AI-generated content. The generalized algorithm for the AI Summary Generation module is as follows:

- **Step 1 (Input Collection):** The user inputs their target "Job Title" and "Core Skills" in the frontend form.
- **Step 2 (Prompt Engineering):** The frontend constructs a predefined, highly optimized prompt string. For example: "Generate 3 distinct, professional resume summaries for a [Job Title] with skills in [Core Skills]. Keep them concise and ATS-friendly."
- **Step 3 (API Request):** An asynchronous HTTP POST request is triggered to the OpenRouter API endpoint containing the generated prompt and authentication headers.
- **Step 4 (Loading State):** The UI enters a loading state, disabling the "Generate" button to prevent duplicate requests and displaying a visual loader.
- **Step 5 (Data Processing):** The backend/API returns a JSON response containing the generated text. The algorithm parses this JSON to extract the 3 individual summary options.
- **Step 6 (UI Update):** The loading state is cleared, and the 3 summaries are rendered in selectable UI cards.
- **Step 7 (Selection & Injection):** The user clicks their preferred summary, which is instantly injected into the active resume state and rendered on the Live Preview.

3.2 Function Oriented Design for Procedural Approach

The system is divided into several decoupled, functional modules to ensure maintainability:

- **Authentication Module:** Handles user sessions (if implemented) or anonymous session IDs for local storage persistence.

- **Form Management Module:** Contains sub-functions for handling input changes (handleChange), adding new list items (addExperience, addEducation), and deleting list items (removeSkill).
- **AI Integration Module:** Contains pure functions dedicated to communicating with the OpenRouter API, handling errors (e.g., timeout, rate limits), and parsing the response.
- **Preview Render Module:** Listens to the global state (using React Context or Redux) and re-renders the DOM elements in the preview pane whenever the state changes.
- **PDF Export Module:** Converts the HTML structure of the Preview Render Module into a downloadable PDF format using the browser's native print API or libraries like html2pdf.js.

3.3 System Design

Table 3.1: Core modules and responsibilities

Module Name	Responsibility
Form Management Module	Handles all user input changes, adding new dynamic list items (like Experience or Education), and deleting items.
AI Integration Module	Responsible for communicating with the OpenRouter API, managing prompts, handling network timeouts, and parsing AI text responses.
Preview Render Module	Listens to the global application state and re-renders the DOM elements in the preview pane whenever user data changes.
PDF Export Module	Captures the HTML/CSS structure of the Live Preview and converts it into a standardized, downloadable PDF format.

The system follows a modern Client-Server architecture, where the React-based frontend acts as the primary client, communicating with third-party APIs (OpenRouter) for AI functionalities.

3.3.1 Data Flow Diagrams (Level 0, Level 1)

- **Level 0 DFD (Context Diagram):** Shows the "Smart Resume Crafter System" as a single central circle. Arrows point from the "User" entity to the system (labeled "User Details, API Requests") and arrows point from the system back to the user (labeled "Live Preview, AI Content, PDF Download"). Another entity, "OpenRouter API", receives "Prompt Context" and returns "Generated Text Response".

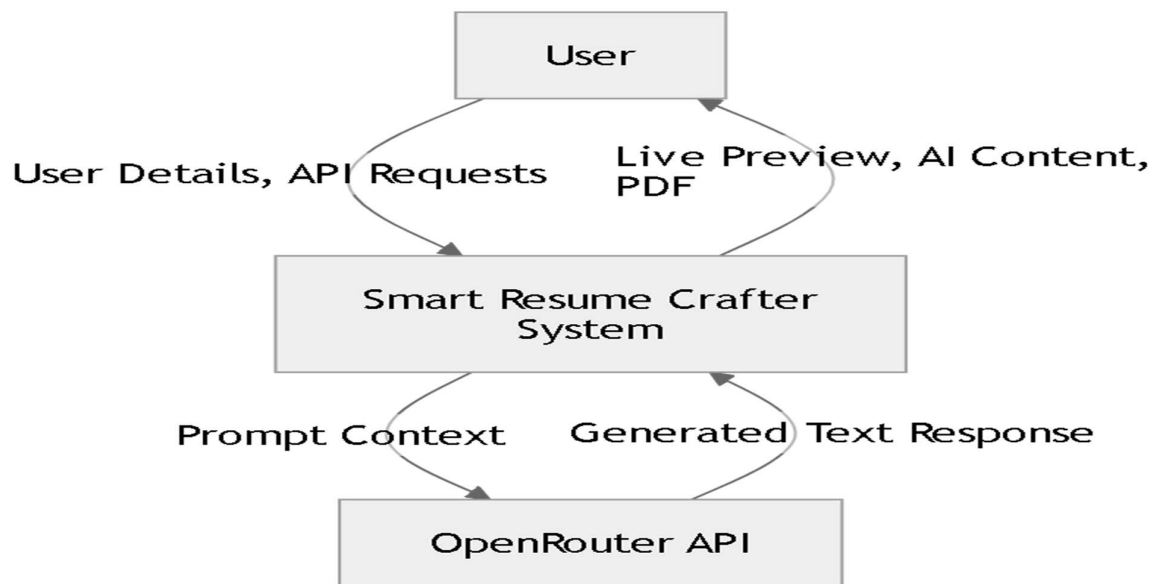


Fig 3.3.1: Data Flow Diagram (Level 0)

Level 1 DFD: Breaks the system down into four processes:

- Input Management: Accepts user data.
- AI Processing: Sends data to the external API and formats the response.
- Live Rendering: Updates the visual resume layout.
- Document Export: Packages the final view into a PDF.

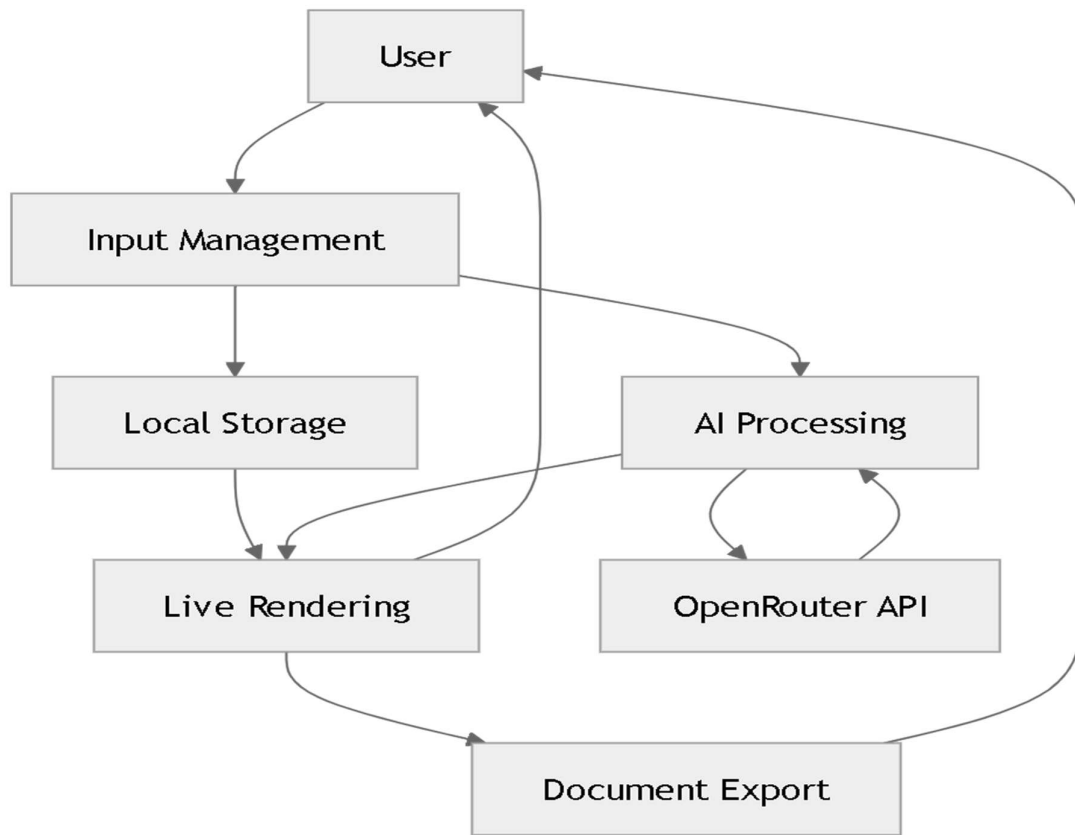


Fig 3.3.2: Data Flow Diagram (Level 1)

3.3.2 Activity Diagram

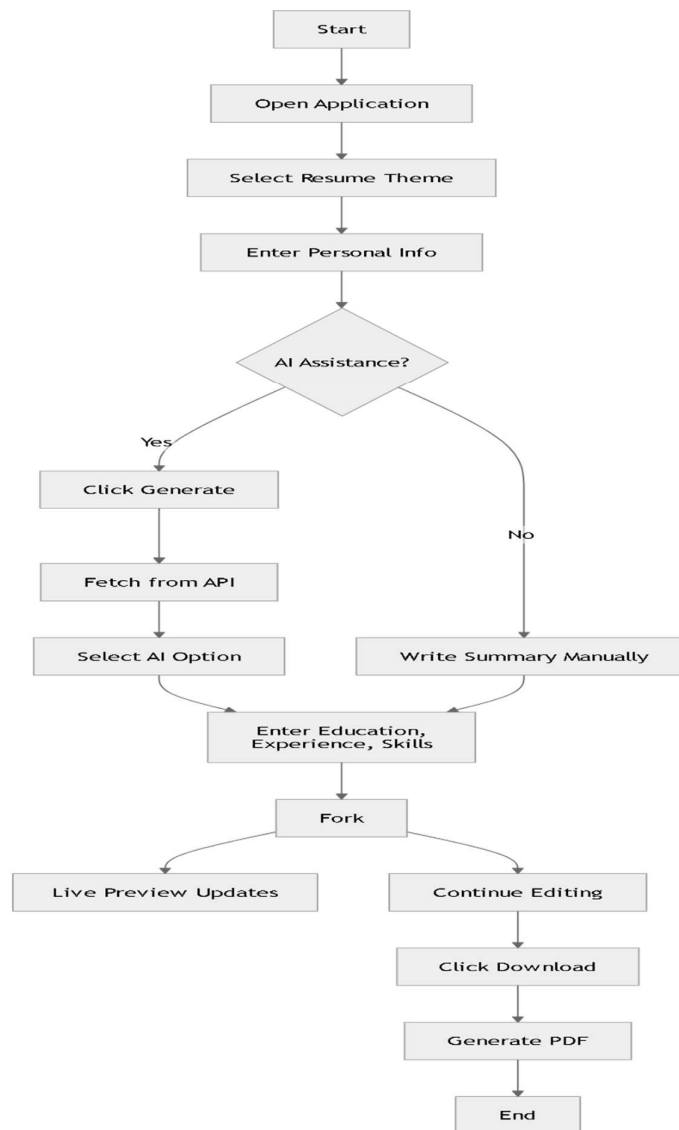


Fig 3.3.3: Activity Diagram

The Activity Diagram: illustrates the workflow of the user while interacting with the application. It begins with the user opening the application and selecting a resume template. The user then enters personal details and decides whether to use AI assistance.

If AI is used, the system generates suggestions via API; otherwise, the user manually inputs data. The process continues with adding education, experience, and skills while the live preview updates simultaneously. Finally, the user downloads the resume as a PDF.

3.3.3 Flow Chart

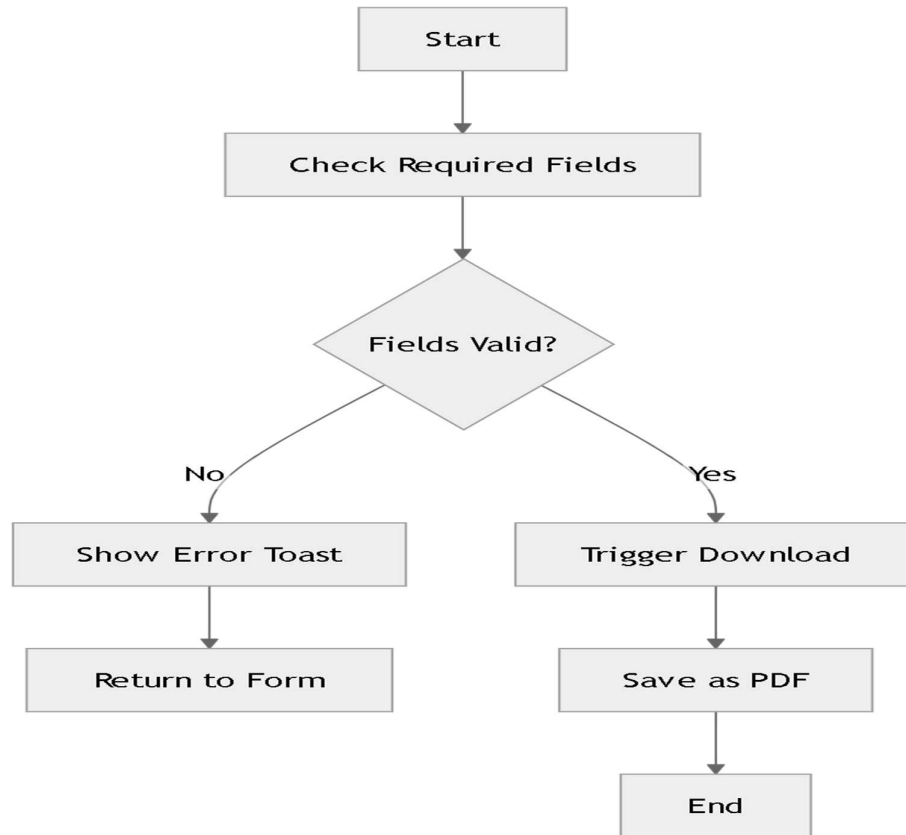


Fig 3.3.4: Flow Chart

The Flowchart: represents the validation and download process of the resume. It starts by checking whether all required fields such as name, email, and phone are filled correctly.

If any field is missing, the system displays an error and redirects the user back to editing. If all inputs are valid, the system proceeds to generate and download the resume in PDF format. This ensures data accuracy before final output.

3.3.4 Class Diagram

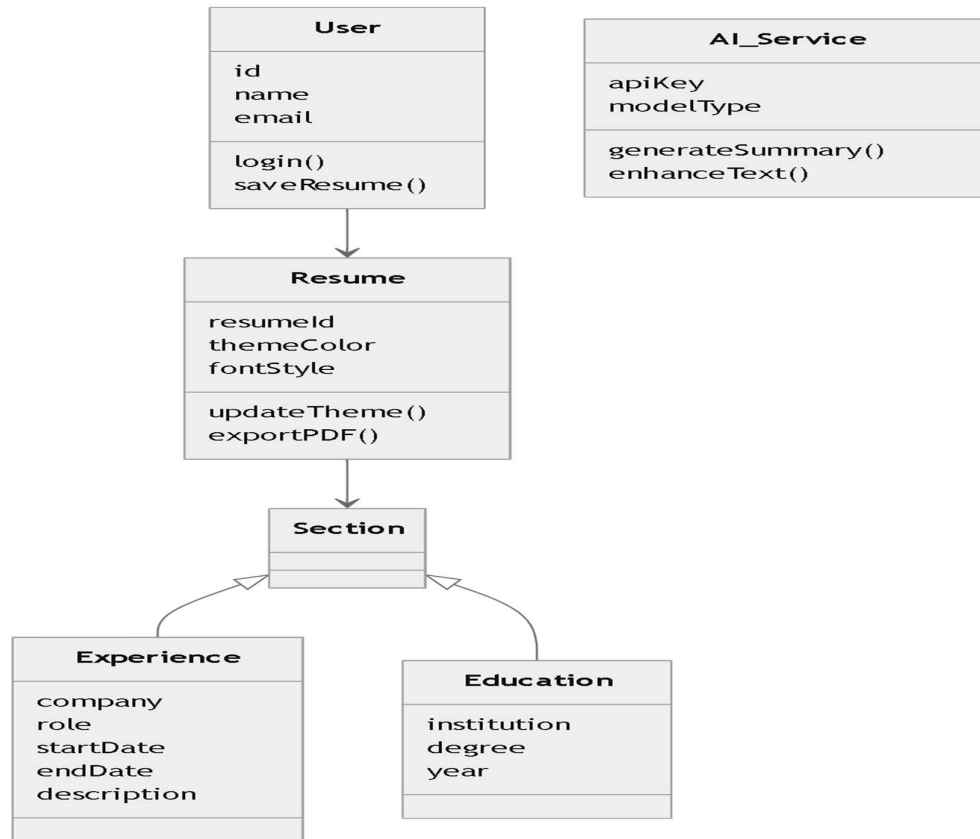


Fig 3.3.5: Class Diagram

The Class Diagram: represents the structure of the system by showing different classes, their attributes, and methods. The main classes include User, Resume, Section, Experience, Education, and AI_Service.

It also shows relationships such as inheritance (Experience and Education from Section) and associations between User and Resume. This diagram helps in understanding the object-oriented design of the application.

3.3.5 ER Diagram

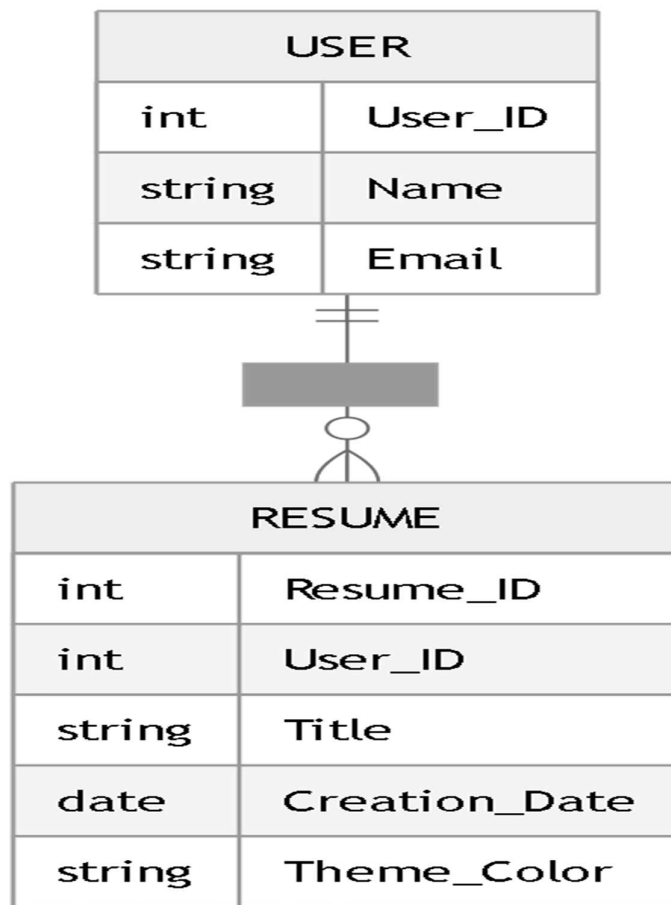


Fig 3.3.6: Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) Diagram: defines the logical structure of data storage in the system. It shows entities such as USER and RESUME along with their attributes.

The relationship between these entities is one-to-many, meaning one user can create multiple resumes. This diagram helps in visualizing how data is organized and managed within Local Storage or a database.

3.3.6 Sequence Diagram

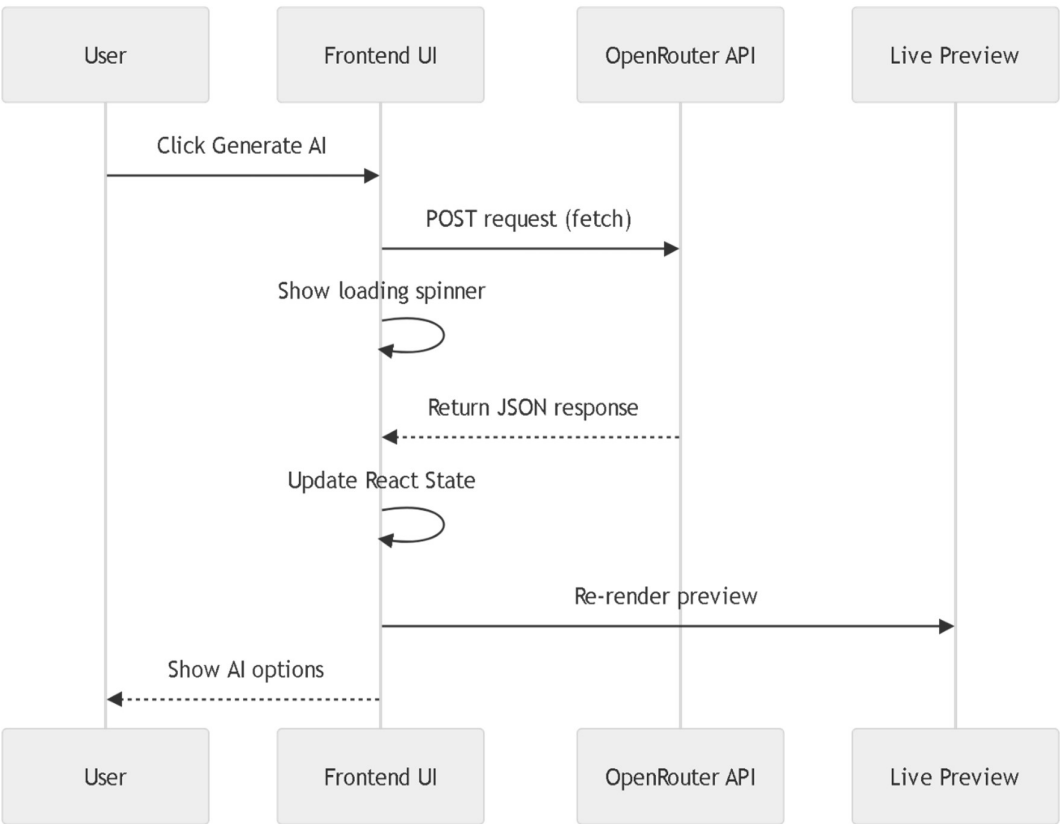


Fig 3.3.7: Sequence Diagram

The Sequence Diagram: illustrates the interaction between different components during AI content generation. It shows how the User interacts with the Frontend UI, which sends a request to the OpenRouter API.

The API processes the request and returns a response, which updates the system state and refreshes the live preview. This diagram highlights the step-by-step communication flow in the system.

3.4 Database Design

The system utilizes **MongoDB**, a NoSQL document-oriented database, to store user credentials and resume data. Database interactions are handled using **Mongoose (ODM)**, which provides a schema-based solution to model application data in Node.js.

MongoDB is chosen due to its flexibility in handling complex and hierarchical data structures such as resumes, which contain nested fields like experience, education, and skills.

3.4.1 Logical Database Design

The system follows a NoSQL document-oriented approach where data is stored in collections as JSON-like documents. Relationships between entities are managed using references.

- **Collection: users**
 - `_id` (ObjectId)
 - `fullName` (String)
 - `email` (String)
 - `password` (Hashed String)
- **Collection: resumes**
 - `_id` (ObjectId)
 - `user` (ObjectId reference to users)
 - `personalInfo` (Object)
 - `firstName`, `lastName`, `email`, `phone`, `address`, `jobTitle`, `summary`, `themeColor`
 - `experience` (Array of Objects)
 - `education` (Array of Objects)
 - `skills` (Array of Objects)
 - `projects` (Array of Objects)
 - `certifications` (Array of Objects)

This design supports a **one-to-many relationship**, where one user can create multiple resumes.

3.4.2 Physical Database Design

The physical implementation uses BSON (Binary JSON) format through Mongoose schemas.

- **Data**

- **Types:**

Strings, Numbers, Booleans, Arrays, and ObjectIds are used for efficient storage and retrieval.

- **Indexes:**

- `_id` fields are indexed by default
- email field is marked as **unique** to prevent duplicate accounts

- **References:**

The `user` field in the `resumes` collection acts as a foreign key. Mongoose's `populate()` method is used to fetch related user data efficiently.

This design ensures **scalability, fast querying, and structured data handling**.

Chapter 4 – Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDE's, Tools and Technologies used

The implementation of the Smart Resume Crafter utilized a modern, robust technology stack tailored for high-performance web applications:

- **Frontend Framework (React.js via Vite):** React was chosen for its component-based architecture, allowing for reusable UI elements (e.g., repeating experience forms). Vite was used as the build tool due to its extremely fast Hot Module Replacement (HMR) during development.
- **Styling (Tailwind CSS / Vanilla CSS):** Tailwind CSS was utilized to rapidly build custom, responsive designs without leaving the HTML/JSX files. It enabled the creation of the dark-themed "Luminary" aesthetic efficiently.
- **Icons (Lucide-React):** Used for lightweight, scalable vector icons across the interface.
- **AI API (OpenRouter):** OpenRouter was chosen as the AI gateway, allowing the application to interface seamlessly with advanced models like Gemini for text generation tasks.
- **IDE (Visual Studio Code):** VS Code served as the primary Integrated Development Environment, supported by extensions for ESLint (code quality) and Prettier (code formatting).
- **Version Control (Git & GitHub):** Used for source code management, tracking changes, and maintaining a repository of the project timeline.

4.2 Testing Techniques and Test Plans

Table 4.1: Tools and technologies used

Category	Technology Used	Purpose
Frontend Framework	React.js (via Vite)	Component-based architecture for building reusable UI elements; Vite used for fast development builds.
Styling	Tailwind CSS	Utility-first CSS framework used to rapidly build custom, responsive designs and the "Luminary AI" theme.
AI Service / API	OpenRouter API (Gemini)	The core intelligence engine used to generate professional summaries and experience descriptions.
Icons & Assets	Lucide-React	Provides lightweight, scalable vector icons across the user interface.

To ensure the reliability and usability of the application, several testing methodologies were employed throughout the development lifecycle.

- **Unit Testing:** Individual components, particularly the utility functions responsible for formatting dates and parsing the AI API responses, were tested in isolation to ensure they handle edge cases (e.g., empty strings, null API responses) gracefully.
- **Integration Testing:** The interaction between the React forms and the global state was heavily tested. Specifically, ensuring that adding or deleting an "Experience" block in the form accurately updates the Live Preview without causing infinite re-renders.
- **User Interface (UI) Testing:** The application was tested across multiple screen resolutions (Desktop, Tablet, Mobile) using Chrome DevTools to verify that the Tailwind CSS responsive classes (like md:flex-row, sm:block) behaved correctly.

- **API Stress Testing:** The OpenRouter API connection was tested for timeout handling. Simulated network delays were introduced to verify that the loading spinners appeared and that the system did not crash if the API failed to respond.
- **End-to-End (E2E) Testing (Manual):** A complete user flow was simulated: A user opens the app -> fills out personal details -> uses AI to generate a summary -> adds education -> changes the theme color -> and successfully downloads the final PDF.

4.3 Installation Instructions

Table 4.2: Testing summary

Testing Phase	Focus Area	Expected Outcome
Unit Testing	Utility functions (date formatting, API parsing)	Functions successfully handle edge cases, empty strings, and null responses without crashing.
Integration Testing	React forms & Global State	Adding or deleting form items accurately updates the Live Preview without causing infinite re-renders.
UI Testing	Mobile, Tablet, and Desktop resolutions	Tailwind CSS responsive classes adapt perfectly, ensuring usability on smaller screens.
API Stress Testing	OpenRouter API timeout handling	The system displays loading spinners properly and gracefully handles failed API responses.
End-to-End Testing	Complete user journey	A user can fill forms, use AI, change themes, and download a PDF successfully from start to finish.

To set up and run the Smart Resume Crafter locally, the following steps are required:

- **Prerequisites:** Ensure Node.js (v16.x or higher) and npm (Node Package Manager) are installed on the local machine.
- **Clone Repository:** Download the source code or clone the repository via Git: `git clone [repository_url]`
- **Navigate to Directory:** Open a terminal and navigate to the frontend folder: `cd Frontend`
- **Install Dependencies:** Run the command `npm install` to download all required packages (React, Vite, Tailwind, etc.) listed in the `package.json` file.
- **Environment Variables:** Create a `.env` file in the root directory and add the necessary API keys (e.g., `VITE_OPENROUTER_API_KEY=your_key_here`).
- **Start Development Server:** Run `npm run dev`. The application will compile and launch on a local port (typically `http://localhost:5173`).

4.4 End User Instructions

- **Access the Platform:** Open a modern web browser and navigate to the application URL.
- **Select a Theme:** Choose a preferred primary color for the resume using the theme selector tool.
- **Input Data:** Fill out the forms step-by-step (Personal Details, Summary, Experience, etc.).
- **Use AI Assistance:** In the Summary or Experience sections, click the "Generate with AI" button. Wait for the loading animation to complete, then click on the suggested text you prefer.
- **Review Live Preview:** Observe the right side of the screen to see exactly how the resume looks.
- **Download:** Once satisfied, click the "Download" or "Print" button. In the browser's print dialog, ensure "Save as PDF" is selected and "Background Graphics" is enabled to preserve the theme colours.

Chapter 5 –Results and Discussions

5.1 User Interface Representation

The User Interface (UI) of the Smart Resume Crafter was successfully developed according to the initial design specifications. The application features a split-screen layout designed for maximum productivity. The left pane houses the dynamic, scrollable forms where users input their data, while the right pane remains stationary (or scrolls independently), displaying a high-fidelity, real-time preview of the generated resume document. The aesthetic follows a modern, premium "Luminary AI" theme, utilizing deep purples, crisp white backgrounds for the document preview, and subtle glassmorphic effects on form elements. This professional design instills confidence in the user and ensures the final output looks spectacular.

5.2 Brief Description of Various Modules of the System

Table 5.1: User interface components

UI Component	Description
Split-Screen Layout	The core layout featuring scrollable input forms on the left pane and a stationary, real-time resume document preview on the right pane.
Personal Details Form	Input fields for First Name, Last Name, Email, Phone, and Job Title that bind instantly to the resume header.
AI Action Buttons	"Generate with AI" buttons located in the Summary and Experience sections to trigger OpenRouter API requests.
Theme Customizer	A color selection tool that allows users to instantly change the accent colors and borders of the resume preview.
Export Toolbar	Contains the "Print/Download" button, initiating the conversion of the preview pane into a PDF file.

The final product consists of several fully functional modules:

- **The Editor Module:** This is the core interaction point. It contains sub-sections for Personal Details, Summary, Experience, Education, and Skills. It correctly handles complex state changes, such as reordering skills or adding multiple past jobs.
- **The AI Generator Module:** Successfully integrates with the OpenRouter API. It takes user context, fires the request, and presents three highly professional, ATS-optimized summaries or experience bullet points.
- **The Live Preview Engine:** Acts as a mirror to the Editor Module. It applies the selected CSS theme (e.g., color accents, font families) and formats the raw text into a standard, single-page or multi-page resume layout.
- **The PDF Export Module:** Uses browser rendering techniques to perfectly capture the DOM elements of the Live Preview and convert them into a standard A4 or Letter-sized PDF file, completely preserving the CSS styling.

5.3 Snapshots of System with Brief Detail of Each

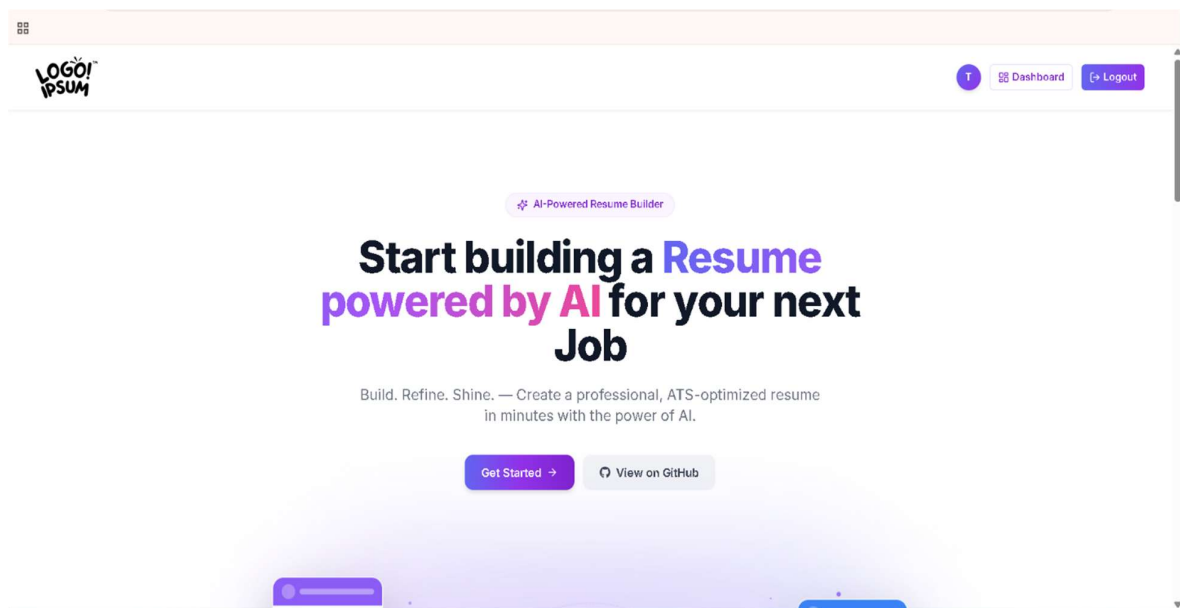


Fig 5.1: The Landing Page

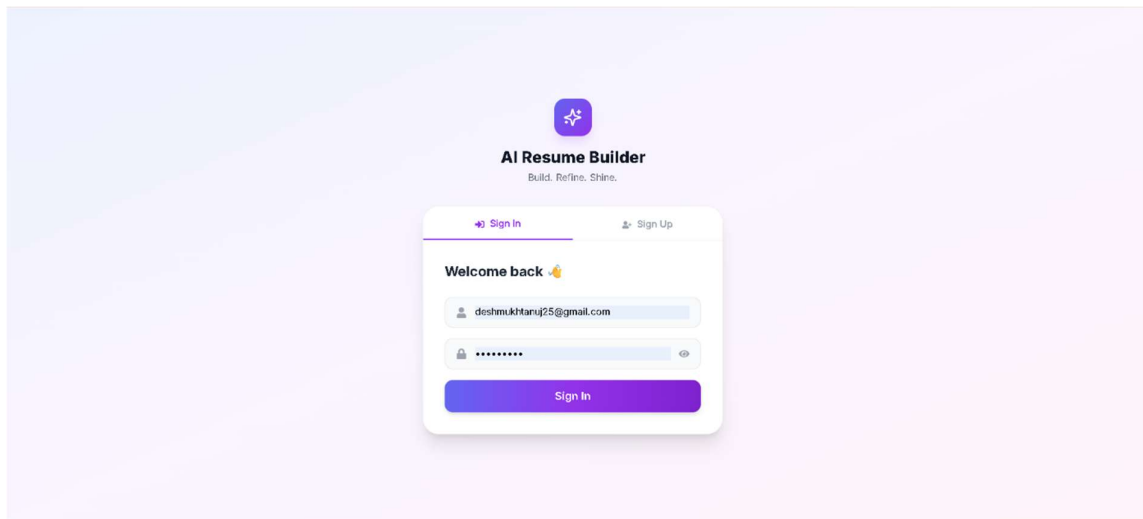


Fig 5.2: Login Page

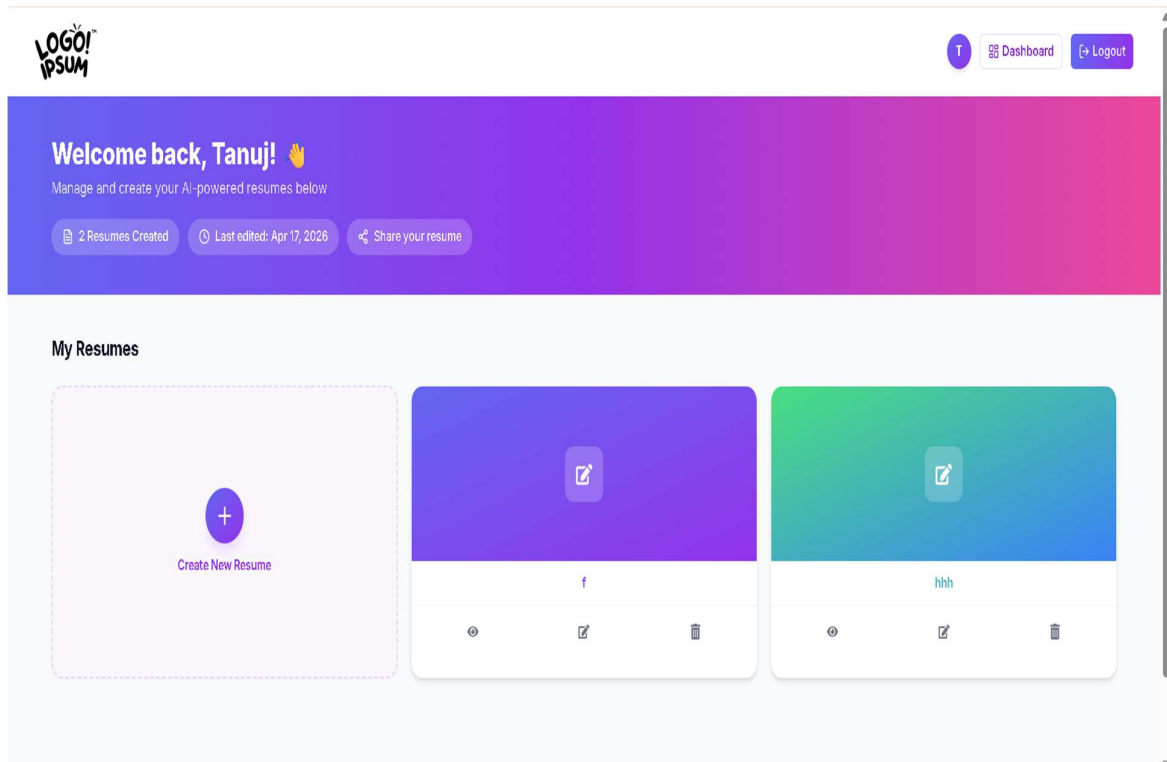


Fig 5.3: Dashboard

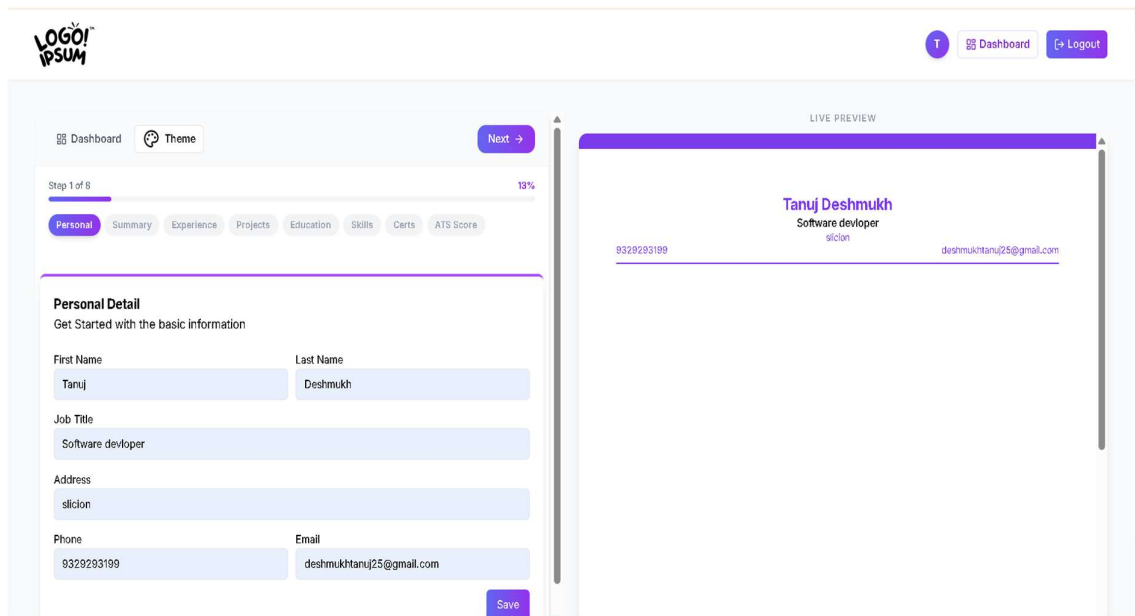


Fig 5.4: The Split-Screen Editor

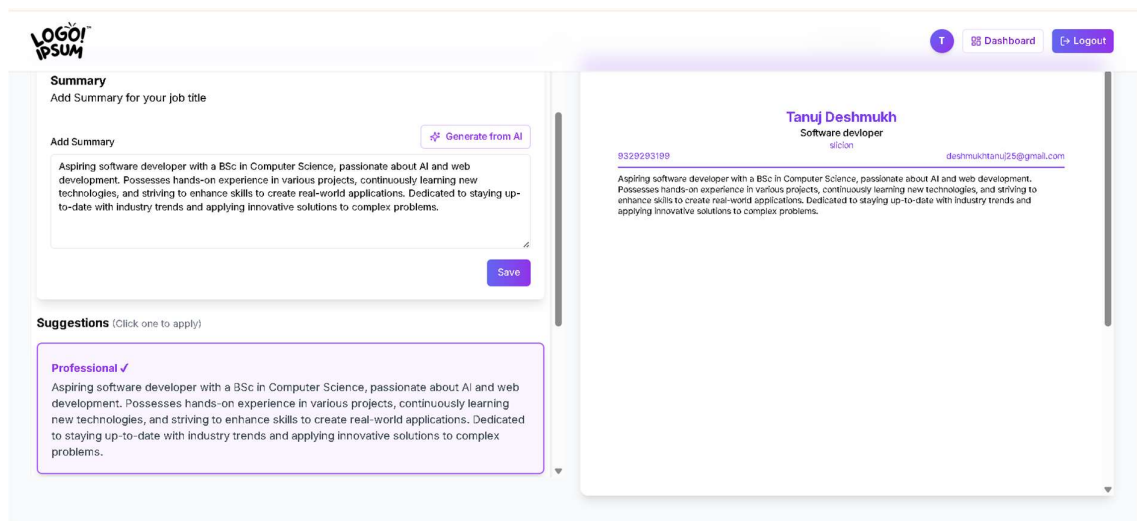


Fig 5.5: AI Summary Generation

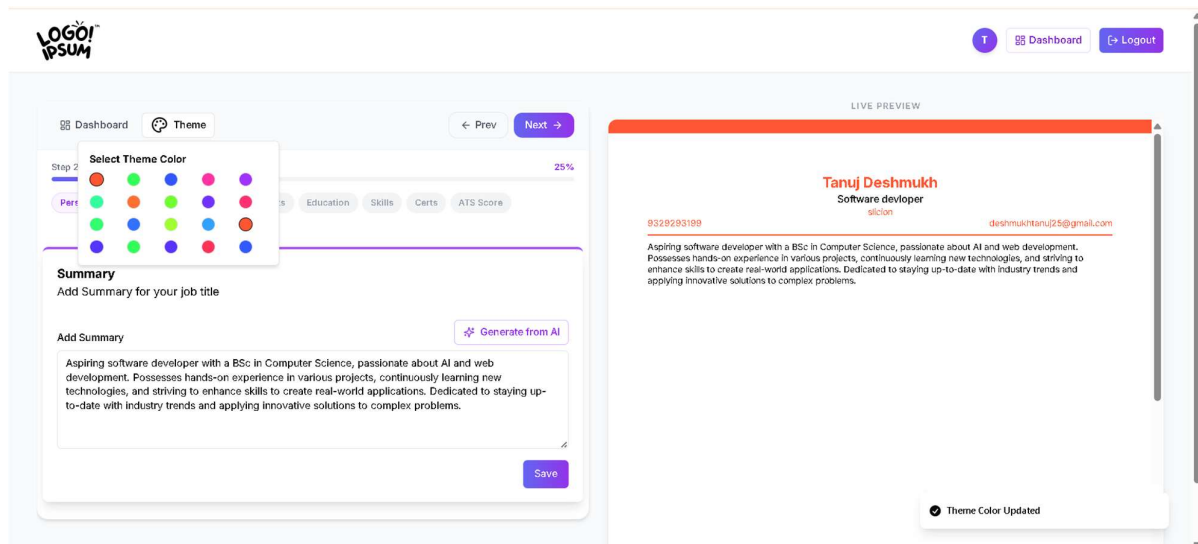


Fig 5.6: Theme Customization

5.4 Back Ends Representation (Database to be used)

The backend of the Smart Resume Crafter is implemented using MongoDB, a NoSQL document-oriented database. The database stores user profiles and resume data in structured JSON-like documents. Each user can create and manage multiple resumes, with data including personal information, experience, education, skills and other related sections.

MongoDB provides flexibility in handling nested and dynamic data structures, making it suitable for resume storage. The database ensures efficient data retrieval, scalability, and secure storage of user information. The interaction with the database is handled using Mongoose, which provides schema-based modeling in Node.js.

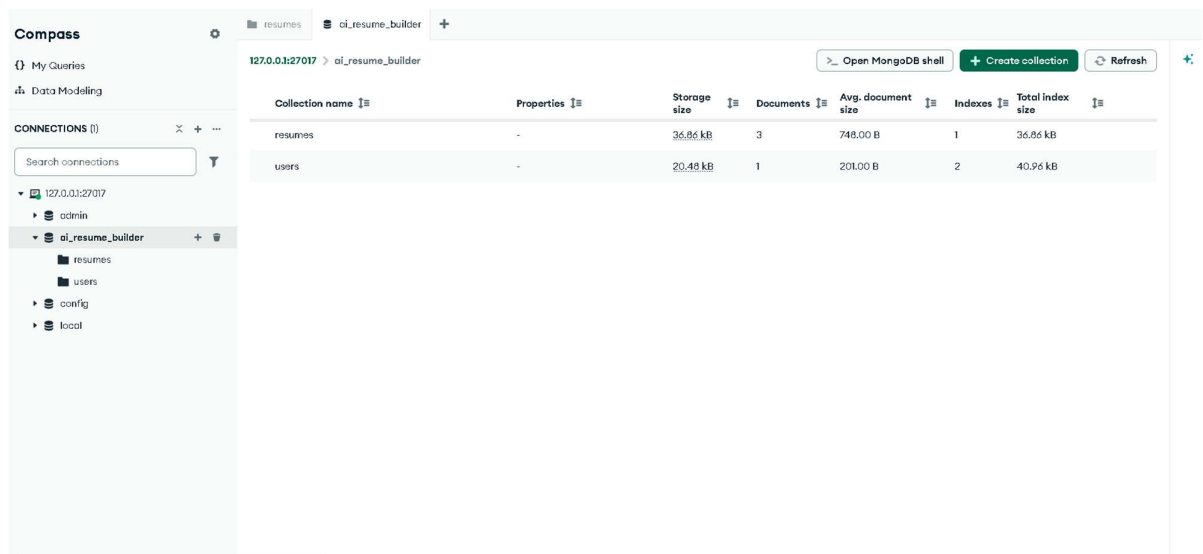


Fig 5.7: MongoDB Database Structure

5.5 Snapshots of Database Tables with Brief Description:

Table 5.2: Database and local storage structure

DataObject (Key)	Data Type	Description
user Id	String	A unique identifier for the user's session.
Personal Info	Object	Stores key-value pairs for firstName, lastName, email, phone, and jobTitle.
experience	Array of Objects	Stores multiple past jobs, including fields for company, role, startDate, endDate, and description.
education	Array of Objects	Stores academic history, including fields for institution, degree, and graduationYear.
skills	Array of Strings	A structured list containing the technical and soft skills entered by the user.

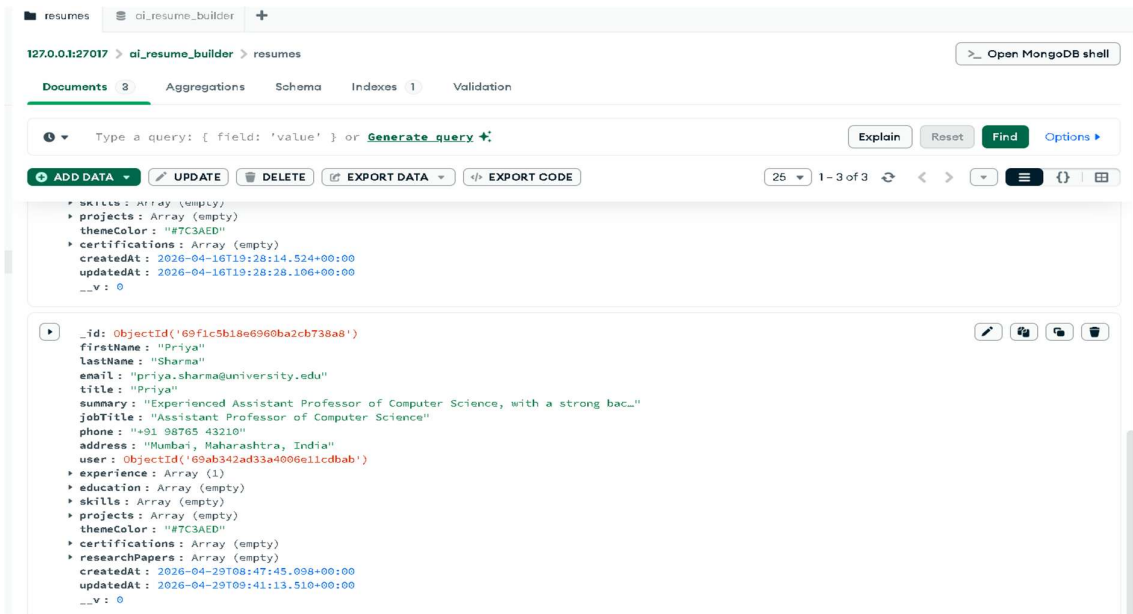


Fig 5.8: MongoDB Resume Collection

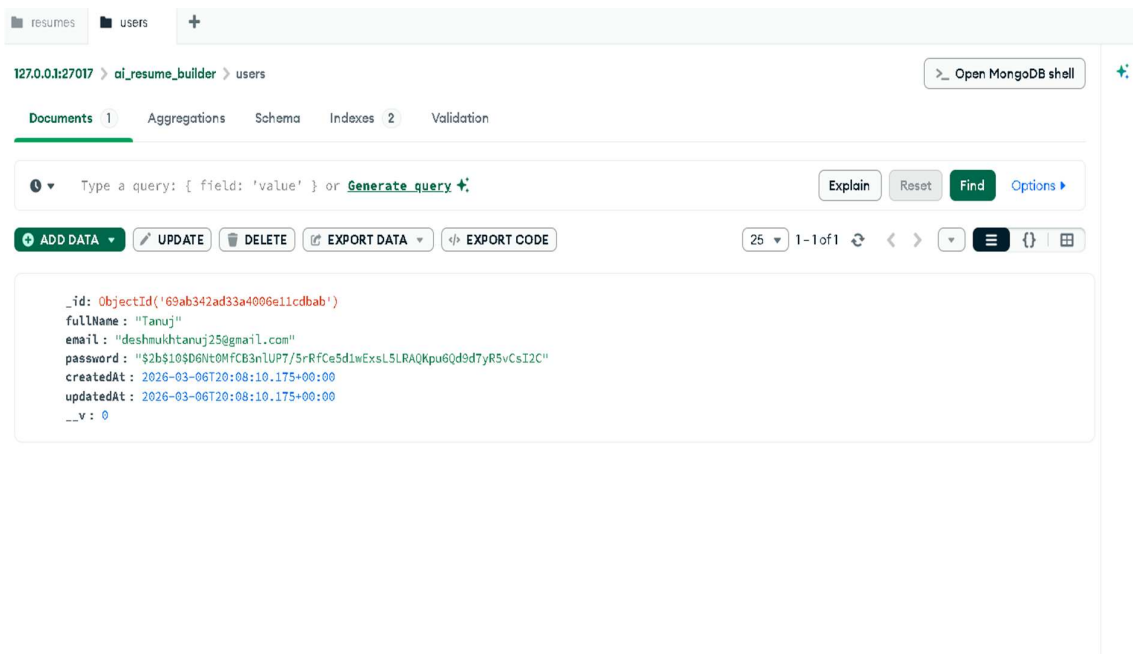


Fig 5.9: MongoDB User Collection

Chapter 6-Summary and Conclusions

6.1 Summary

The “Luminary AI – Smart Resume Builder with Real-Time AI Suggestions & Live Preview” project successfully achieved its primary objective of simplifying and enhancing the resume creation process using artificial intelligence and modern web technologies.

By identifying the limitations of traditional word processors and basic template-based resume builders, the system provides a seamless and user-friendly interface with a real-time live preview feature. This enables users to visualize changes instantly while editing their resumes, eliminating the need for repeated formatting and adjustments.

The integration of the OpenRouter API, utilizing advanced AI models, enables automatic generation of professional summaries and experience descriptions. This feature significantly improves the quality of resume content by providing context-aware and well-structured suggestions, especially beneficial for users with limited writing experience.

From a technological perspective, the system is developed using React.js and Tailwind CSS for the frontend, ensuring a responsive and visually appealing interface. The backend is implemented using Node.js and Express.js, while MongoDB is used as the database to store user and resume data efficiently.

The application supports multiple resume sections such as Personal Details, Summary, Experience, Education, Skills, Projects, Certifications, and Research Papers. Additionally, the PDF export feature allows users to download professionally formatted resumes suitable for real-world job applications.

6.2 Conclusions

The Luminary AI Resume Builder demonstrates the effective application of artificial intelligence in solving real-world problems related to resume creation. The system successfully reduces the complexity involved in designing and writing resumes while improving overall quality and presentation.

The following conclusions can be drawn from the project:

- AI-based content generation reduces the time and effort required to create professional resumes.
- The real-time live preview feature enhances user experience by providing immediate visual feedback.
- The use of modern web technologies ensures high performance, scalability, and responsiveness.
- MongoDB provides an efficient and flexible data storage solution for managing structured and unstructured resume data.

Chapter 7-Future Scope

While the current system is robust and fully functional, there are several avenues for future enhancement and expansion:

1. **Multiple Templates:** Expanding the system to support a wide variety of structural templates (e.g., Creative, Academic CV, Minimalist) beyond just changing theme colors.
2. **Cloud Database Integration:** Transitioning from LocalStorage to a cloud-based database (like Firebase or PostgreSQL) to allow users to create accounts, save multiple different versions of their resumes, and access them from any device.
3. **Cover Letter Generator:** Expanding the AI capabilities to read the user's finished resume and automatically generate a tailored cover letter for a specific job application.
4. **Direct LinkedIn Import:** Allowing users to authenticate with LinkedIn and automatically populate the resume fields using their existing LinkedIn profile data to save even more time.

Appendix

Appendix A: Glossary of Technical Terms

Term	Definition
React.js	A JavaScript library for building user interfaces using reusable components.
Vite	A modern frontend build tool with fast development and hot module replacement.
Tailwind CSS	A utility-first CSS framework used for styling UI elements.
Node.js	A JavaScript runtime for executing server-side code.
Express.js	A web framework for Node.js used to build APIs.
MongoDB	A NoSQL database storing data in JSON-like format.
Mongoose	An ODM library for MongoDB providing schema-based modeling.
MERN Stack	A full-stack framework (MongoDB, Express, React, Node.js).
API	A set of rules enabling communication between systems.
REST API	An API style using HTTP methods like GET, POST, PUT, DELETE.
OpenRouter API	A gateway to access multiple AI models.
LLM	Large Language Model used for generating human-like text.
JSON	A format for storing and exchanging structured data.
BSON	Binary format used by MongoDB.
JWT	A token used for authentication.
bcrypt	A library for hashing passwords securely.
CORS	A mechanism to allow cross-domain requests.

Frontend	The user interface of the application.
Backend	The server-side logic of the application.
Live Preview	Real-time display of resume while editing.
PDF Export	Conversion of resume into downloadable PDF.

Appendix B: To Be Determined (TBD) List

- TBD-1: Selection of optimal hosting platform for frontend and backend deployment.
- TBD-2: Migration of database to cloud (MongoDB Atlas) for cross-device access.
- TBD-3: Addition of multiple resume templates.
- TBD-4: Implementation of ATS score analysis.
- TBD-5: AI-based cover letter generation.
- TBD-6: Development of mobile application version.
- TBD-7: LinkedIn profile integration.
- TBD-8: Multi-language resume support.
- TBD-9: Collaborative editing feature.
- TBD-10: UI/UX improvements based on user feedback.

Bibliography

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [2] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS)*, pp. 6000–6010, 2017.
- [3] A. Aggarwal, M. Gupta, and A. Singh, "Automated Resume Screening System Using Natural Language Processing," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 6, pp. 250–255, 2020.
- [4] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010.
- [5] K. Lei, Y. Ma, and Z. Tan, "Performance Comparison and Evaluation of Web Development Technologies in PHP, Python, and Node.js," *IEEE International Conference on Computational Science and Engineering*, pp. 661–668, 2014.
- [6] R. Souza, F. Lopes, and T. Silva, "A Comparative Study of NoSQL Database Management Systems for Web Applications," *IEEE Latin America Transactions*, vol. 13, no. 10, pp. 3438–3444, 2015.
- [7] C. Gyorodi, R. Gyorodi, and R. Sotoc, "A Comparative Study of Relational and Non-Relational Database Models in a Web Application," *International Journal of Advanced Computer Science and Applications*, vol. 6, no. 11, pp. 78–83, 2015.
- [8] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," *Doctoral Dissertation*, University of California, Irvine, 2000.
- [9] D. Crockford, "The Application of JavaScript Object Notation (JSON)," *RFC 4627*, IETF, pp. 1–10, 2006.
- [10] A. Mesbah and A. van Deursen, "Migrating Multi-Page Web Applications to Single-Page AJAX Interfaces," *European Conference on Software Maintenance and Reengineering*, pp. 181–190, 2007.

- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171–4186, 2019.
- [12] M. Chen, J. Tworek, H. Jun, et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, pp. 1–32, 2021.